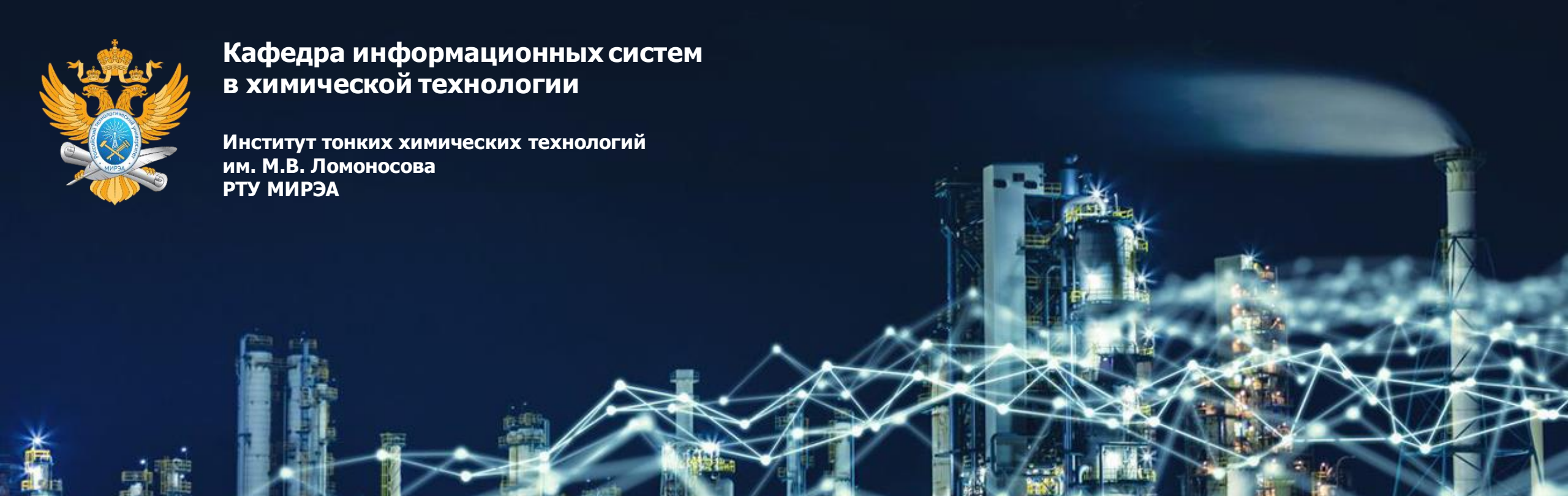




**Кафедра информационных систем
в химической технологии**

**Институт тонких химических технологий
им. М.В. Ломоносова
РТУ МИРЭА**



ИНФОРМАЦИОННЫЕ СИСТЕМЫ В ХИМИЧЕСКОЙ ТЕХНОЛОГИИ



Лекция 10. Утилиты GNU

Содержание лекции

В данной лекции будут рассмотрены следующие темы:

- 🔄 Перенаправление вывода в файл или программу
- 🔄 Редактирование текстовых файлов в командной строке
- 🔄 Поиск и фильтрация текста: `grep`
- 🔄 Регулярные выражения
- 🔄 Поточковые редакторы текста: `sed` и `awk`
- 🔄 Изменение настроек среды командной строки

- 🔄 **Перенаправление вывода в файл или программу**
- 🔄 Редактирование текстовых файлов в командной строке
- 🔄 Поиск и фильтрация текста: `grep`
- 🔄 Регулярные выражения
- 🔄 Поточковые редакторы текста: `sed` и `awk`
- 🔄 Изменение настроек среды командной строки

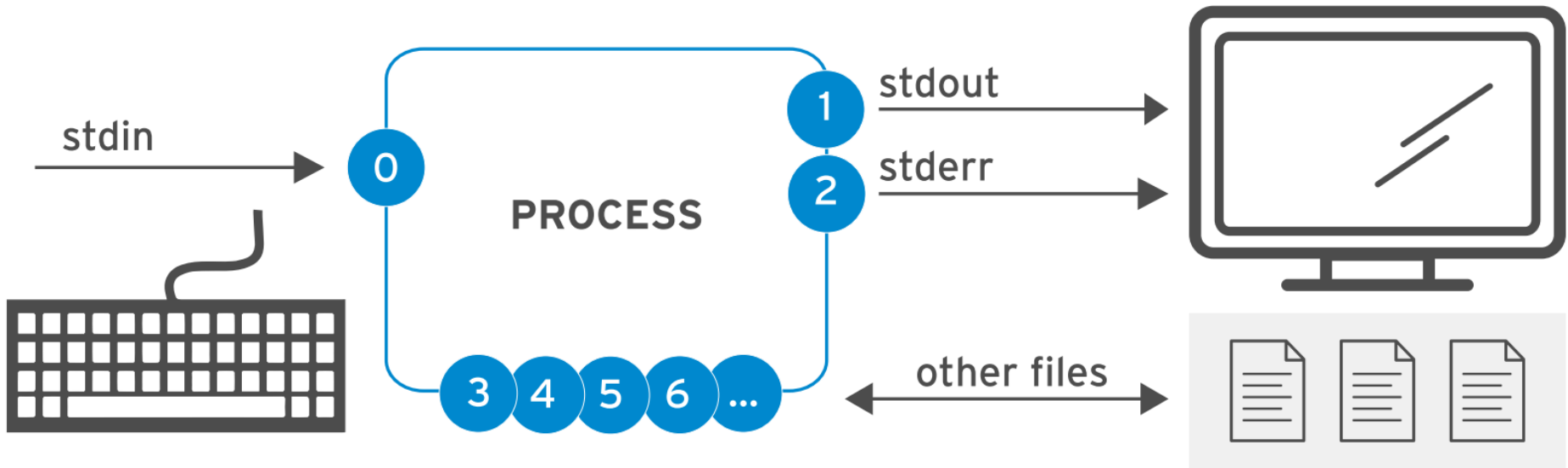
Стандартные потоки ввода и вывода

- Запущенная программа (процесс) в ходе своей работы должна иметь возможность получать и выводить данные
 - Программа, запущенная из командной строки, обычно читает ввод с клавиатуры и выводит данные в окно терминала
- Процесс использует пронумерованные каналы, называемые *дескриптор файла*, для ввода/вывода
 - Стандартный ввод `stdin` (канал 0) читает ввод с клавиатуры
 - Стандартный вывод `stdout` (канал 1) направляет обычный вывод в терминал
 - Стандартный вывод ошибок `stderr` (канал 2) выводит сообщения об ошибках в терминал
- Программа может открывать соединения с другими файлами
 - Используются для ввода и/или вывода
 - Дескрипторы файла получают номера 3 и далее

Стандартные потоки ввода и вывода

Каналы ввода-вывода процесса (дескрипторы файла)

Номер	Имя канала	Описание	Соединение по умолчанию	Применение
0	stdin	Стандартный ввод	Клавиатура	только чтение
1	stdout	Стандартный вывод	Терминал	только запись
2	stderr	Стандартный вывод ошибок	Терминал	только запись
3+	filename	Другие файлы	Нет	чтение и/или запись



Перенаправление ввода/вывода

● Изменяет способ ввода/вывода процесса

- Входные данные могут быть получены из файла, а не с клавиатуры
- Вывод данных или сообщений об ошибках может быть направлен в файл вместо терминала
- Вывод одной программы может быть направлен во ввод другой
- От выходных данных можно избавиться, направив их в **/dev/null**

● При перенаправлении вывода данные и сообщения не отображаются в терминале

- Могут быть записаны в файл для последующего просмотра

● Если вывод перенаправлен в файл

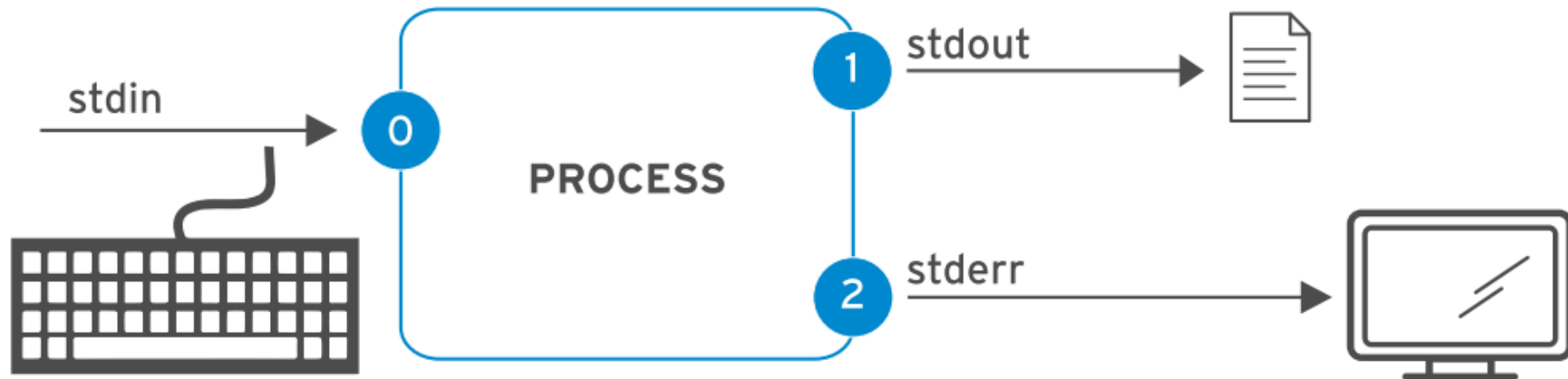
- Если указанный файл не существует, он будет создан
- Если указанный файл существует он будет перезаписан и дополнен данными в зависимости от режима перенаправления

Перенаправление ввода/вывода

Сценарий 1

- Перенаправление стандартного вывода **stdout** с перезаписью файла

```
$ command > file
```

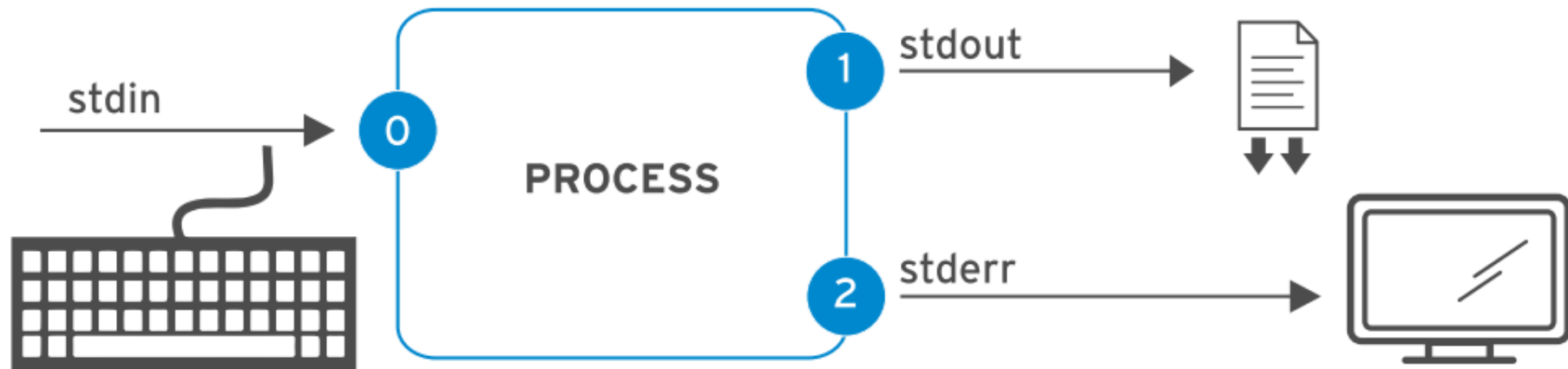


Перенаправление ввода/вывода

Сценарий 2

- Перенаправление стандартного вывода **stdout** с записью в конец файла

```
$ command >> file
```

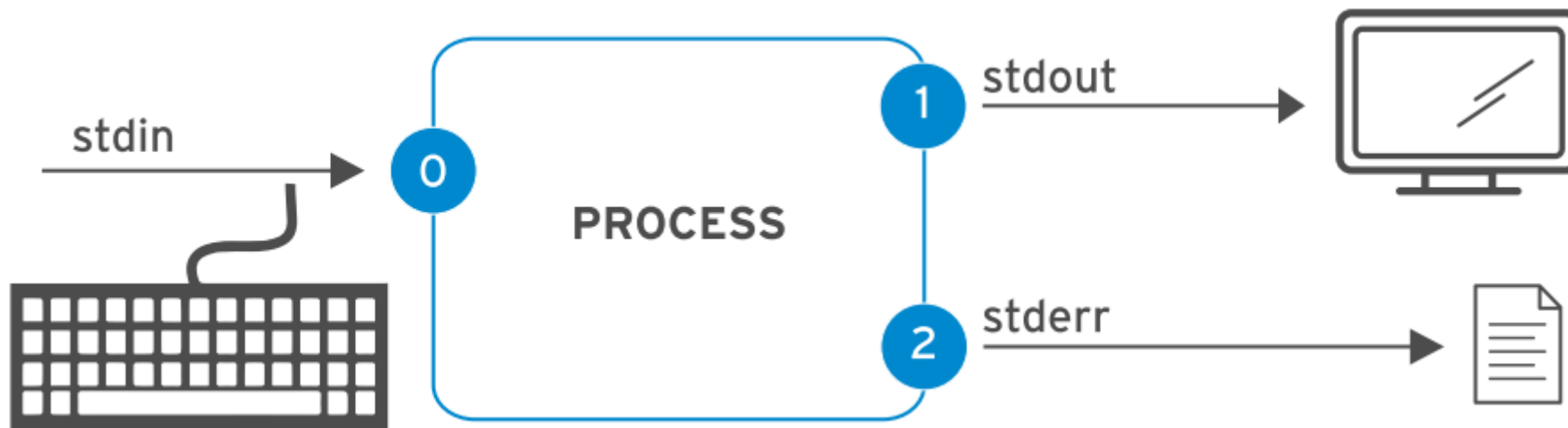


Перенаправление ввода/вывода

Сценарий 3

- Перенаправление стандартного вывода ошибок **stderr** с перезаписью файла

```
$ command 2> file
```

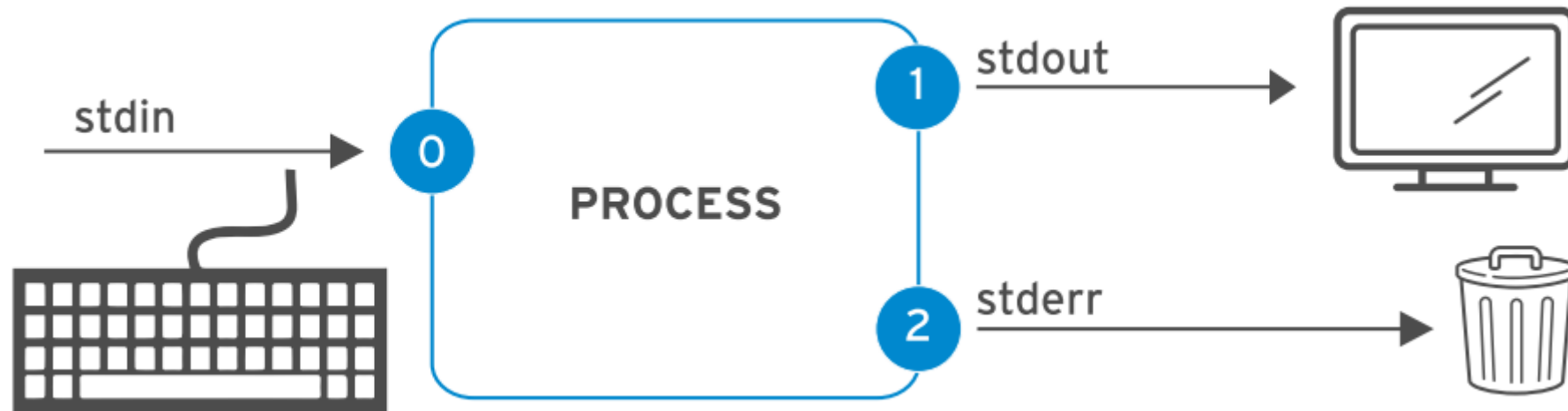


Перенаправление ввода/вывода

Сценарий 4

- Перенаправление стандартного вывода ошибок **stderr** в специальный файл (всегда пустой, данный в нём «исчезают»)

```
$ command 2> /dev/null
```



Перенаправление ввода/вывода

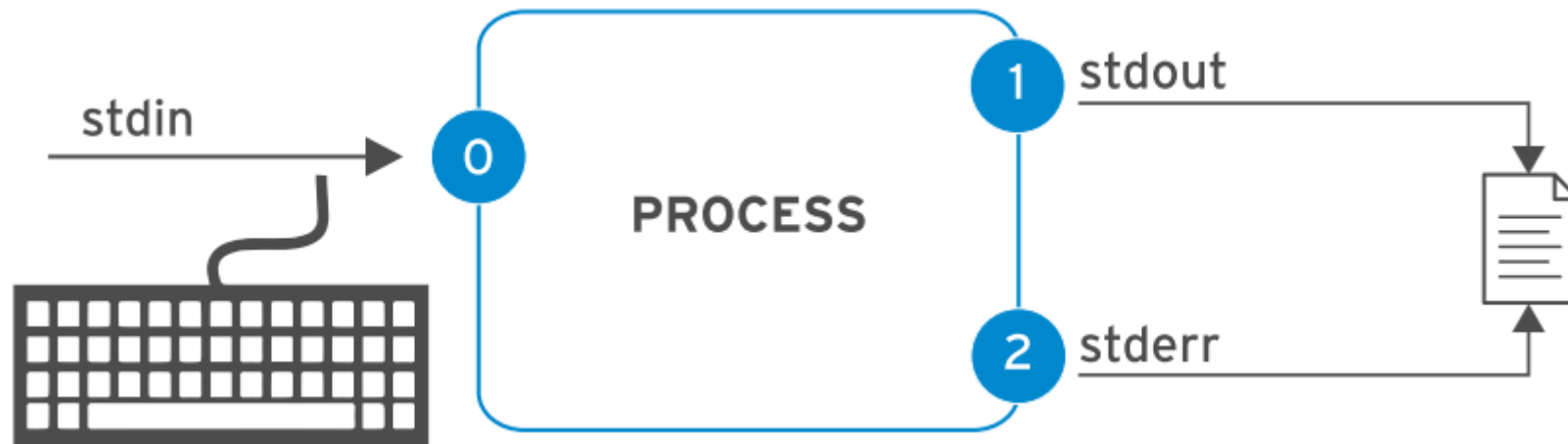
Сценарий 5

- Перенаправление каналов **stdout** и **stderr** в один файл с его перезаписью

или

```
$ command > file 2>&1
```

```
$ command &> file
```



Перенаправление ввода/вывода

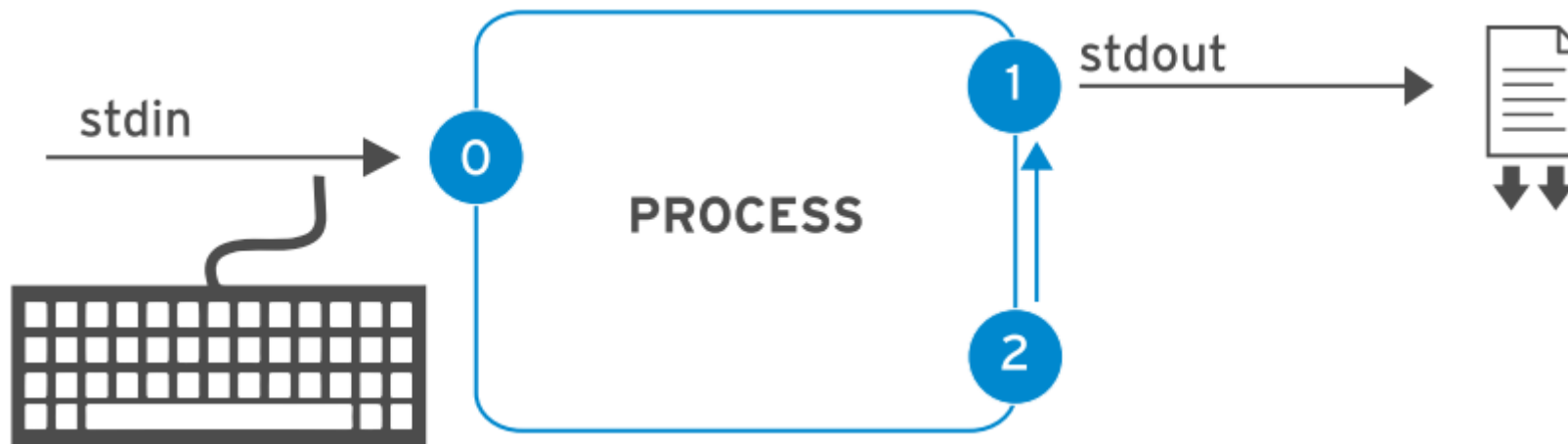
Сценарий 6

- Перенаправление каналов **stdout** и **stderr** в один файл с записью в конец файла

или

```
$ command >> file 2>&1
```

```
$ command &>> file
```



Перенаправление ввода/вывода

Последовательность перенаправления влияет на результат

```
$ command > file 2>&1
```

Стандартный вывод перенаправляется в файл, затем стандартный вывод ошибок перенаправляется туда же, куда стандартный вывод

- Все сообщения попадают в один файл

```
$ command 2>&1 > file
```

Стандартный вывод ошибок перенаправляется туда же, куда стандартный вывод (т.е. в терминал, куда он и так направлен), затем стандартный вывод перенаправляется в файл

- Обычные сообщения записываются в файл, а сообщения об ошибках выводятся в терминал

- Использование синтаксиса `&>file` (или `&>>file`) однозначно направляет весь вывод в один файл, но может не поддерживаться некоторыми `bash`-подобными интерпретаторами командной строки

Перенаправление ввода/вывода

Примеры – нормальный вывод

- Сохранить текущую дату и время

```
$ date > /tmp/saved-timestamp
```

- Скопировать 100 последних строк журнала в другой файл

```
$ tail -n 100 /var/log/dmesg > /tmp/last-100-boot-messages
```

- Объединить содержимое четырех файлов в один

```
$ cat file1 file2 file3 file4 > /tmp/all-four-in-one
```

- Записать список файлов в директории в файл

```
$ ls -a > /tmp/my-file-names
```

- Добавить вывод команд в конец существующего файла

```
$ echo "new line of information" >> /tmp/many-lines-of-information  
$ diff previous-file current-file >> /tmp/tracking-changes-made
```

Перенаправление ввода/вывода

Примеры – вывод сообщений об ошибках

- Вывести обычные сообщения в терминал, об ошибках – записать в файл

```
$ find /etc -name passwd 2> /tmp/errors
```

- Сохранить обычные сообщения и об ошибках в разные файлы

```
$ find /etc -name passwd > /tmp/output 2> /tmp/errors
```

- Объединить содержимое четырех файлов в один

```
$ find /etc -name passwd > /tmp/output 2> /dev/null
```

- Сохранить все сообщения в одном файле

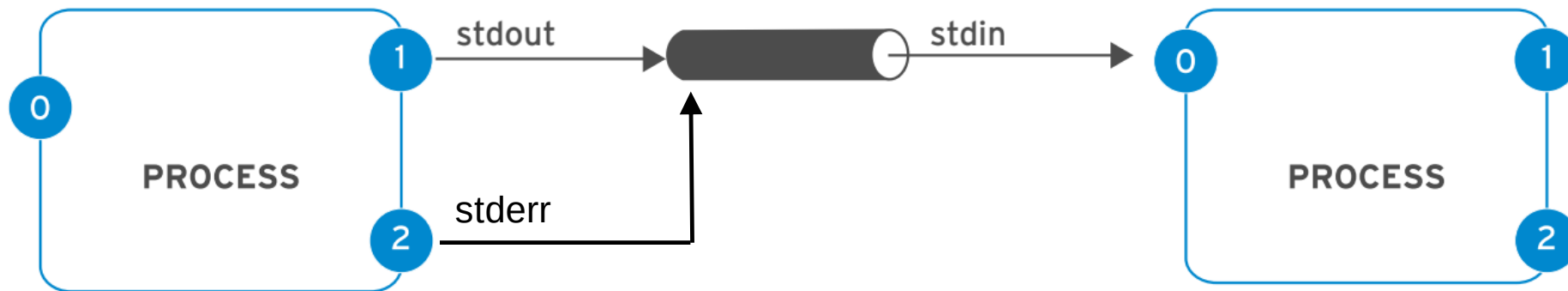
```
$ find /etc -name passwd &> /tmp/save-both
```

- Добавить все сообщения в конец существующего файла

```
$ find /etc -name passwd >> /tmp/save-both 2>&1
```

Создание конвейеров (pipeline)

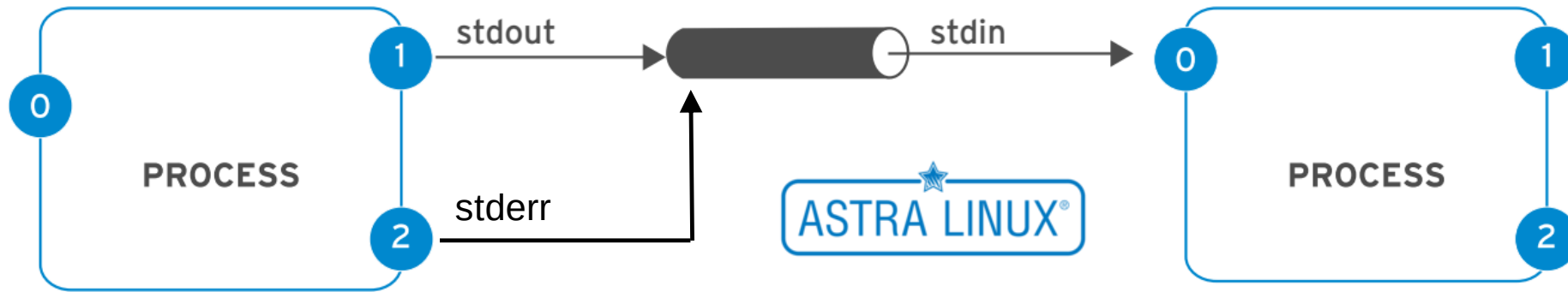
- Конвейер – последовательность из двух или более команд, разделенных символом pipe (|)
 - Соединяет стандартные каналы вывода первой команды со стандартным вводом следующей команды
 - Позволяет дополнительно обрабатывать (например, форматировать или фильтровать) вывод команды другой командой перед выводом его в терминал или файл
 - Создает «поток» данных от команды к команде, позволяя организовывать многоэтапную обработку выходных данных процесса
 - Реализуется как однонаправленный канал межпроцессного взаимодействия
 - Обеспечивает асинхронный запуск команд с параллельным выполнением процессов и буферизацией ввода/вывода



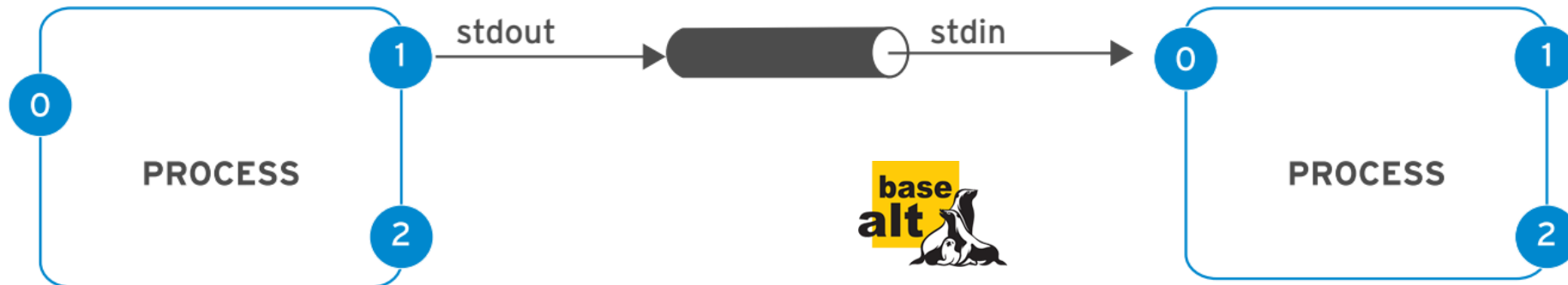
Создание конвейеров (pipeline)

Отличия в дистрибутивах Astra Linux и Alt Linux

- В Astra Linux в конвейер направляется вывод всех выходных каналов



- В Alt Linux в конвейер направляется вывод только канала **stdout**



Создание конвейеров (pipeline)

Примеры

- Организовать отображение вывода команды `ls` постранично

```
$ ls -l /usr/bin | less
```

- Подсчитать количество строк в выводе команды `ls`

```
$ ls | wc -l
```

- Записать в файл первые 10 строк вывода команды `ls`

```
$ ls -t | head -n 10 > /tmp/ten-last-changed-files
```

- Конвейер из нескольких команд

```
$ ls -t | head -n 50 | less
```

- Направить в конвейер оба стандартных выходных канала

```
$ ls -d ./existent_file ./non-existent_file 2>&1 | less  
$ ls -d ./existent_file ./non-existent_file |& less
```



Команда tee

Проблема

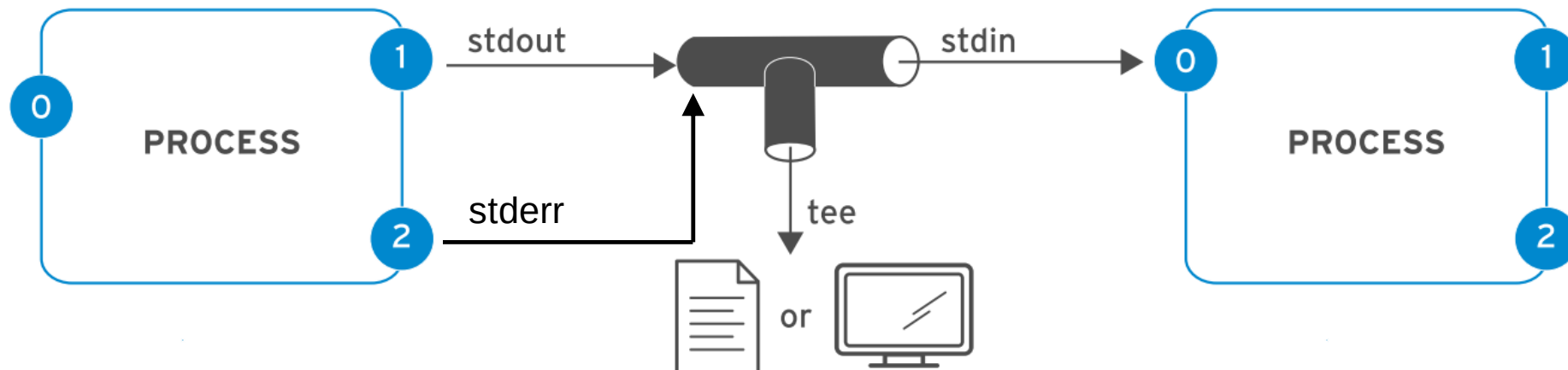
- Если перенаправить вывод команды в файл, в конвейер он уже не попадет
 - В следующем примере вывод `less` будет пустым

```
$ ls > /tmp/saved-output | less
```

- Как перенаправить вывод в файл не нарушая работу конвейера?

Решение

- Использовать команду `tee`
 - Копирует данные из своего стандартного ввода в стандартный вывод и записывает копию данных в указанный файл



Команда tee

Примеры

- Организовать отображение вывода команды ls постранично, сохранив исходный вывод команды в файл

```
$ ls -l | tee /tmp/saved-output | less
```

- Записать в файл первые 10 строк вывода команды ls и вывести их в терминал
 - При использовании в конце конвейера команда tee записывает полученные данные в файл и выводит их копию в терминал

```
$ ls -t | head -n 10 | tee /tmp/ten-last-changed-files
```

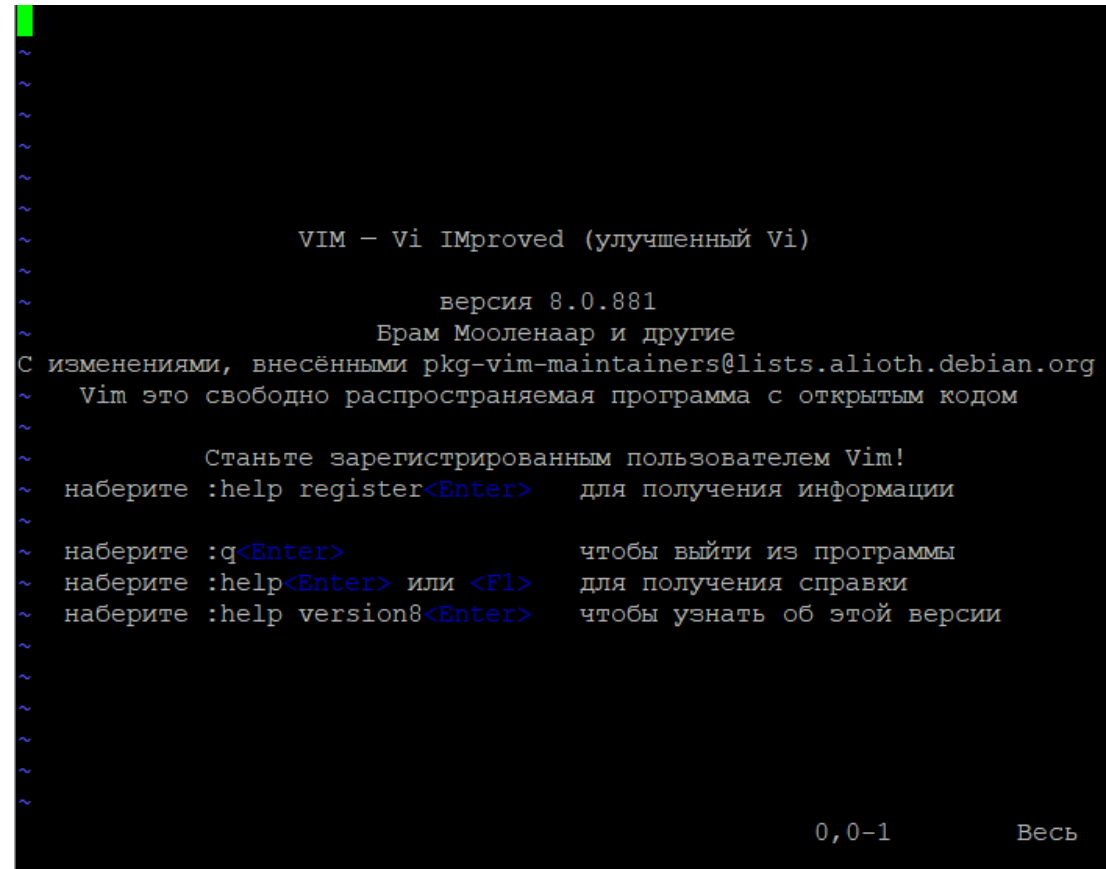
- Перенаправление вывода в файл или программу
- **Редактирование текстовых файлов в командной строке**
- Поиск и фильтрация текста: `grep`
- Регулярные выражения
- Поточковые редакторы текста: `sed` и `awk`
- Изменение настроек среды командной строки

Редактирование текстовых файлов

- Насущная задача для администратора Linux систем
 - Согласно принципам Linux, большинство настроек системы хранятся в текстовых файлах, которые можно просматривать и изменять, используя произвольный простой текстовый редактор
 - Используются как неструктурированные (ini-подобные) так и структурированные (xml, yaml) форматы файлов
- Текстовый редактор для командной строки позволяет не зависеть от наличия графического интерфейса пользователя
 - При управлении серверными системами или при подключении через SSH графический интерфейс будет недоступен
- При выборе текстового редактора для изучения следует ориентироваться на его удобство и распространенность
 - Для Linux систем разработаны десятки текстовых редакторов
 - **Vi** или **Vim** вы встретите почти в любом дистрибутиве Linux

 Vi IMproved создан в 1991 г. и регулярно обновляется

- Основа на vi – текстовый редактор для командной строки, разработанный в 1976 г. для UNIX и, в отличие от современных ему редакторов, поддерживающих визуальный режим редактирования (Visual)
- Реализует функциональность vi во многих современных дистрибутивах Linux, благодаря чему повсеместно распространен
 - POSIX стандарт (частично соблюдается Linux) требует наличия vi в системе
 - Astra Linux и Alt Linux предоставляют VIM
- Предоставляет расширенные возможности, включая режим разделения экрана, цветовую индексацию и подсветку текста при редактировании



Запуск Vim

- Vim может быть установлен в системе в составе разных пакетов и это может влиять на поведение команды запуска

- В дистрибутивах Astra Linux и Alt Linux команды `vi` и `vim` запускают Vim

```
$ vi filename  
$ vim filename
```

- В некоторых дистрибутивах, например, RHEL команда `vi` может запускать упрощенную версию Vim, повторяющую функционал Vi

- Vim – очень мощный текстовый редактор, изучение всей полноты его возможностей требует времени и движения от простого к сложному

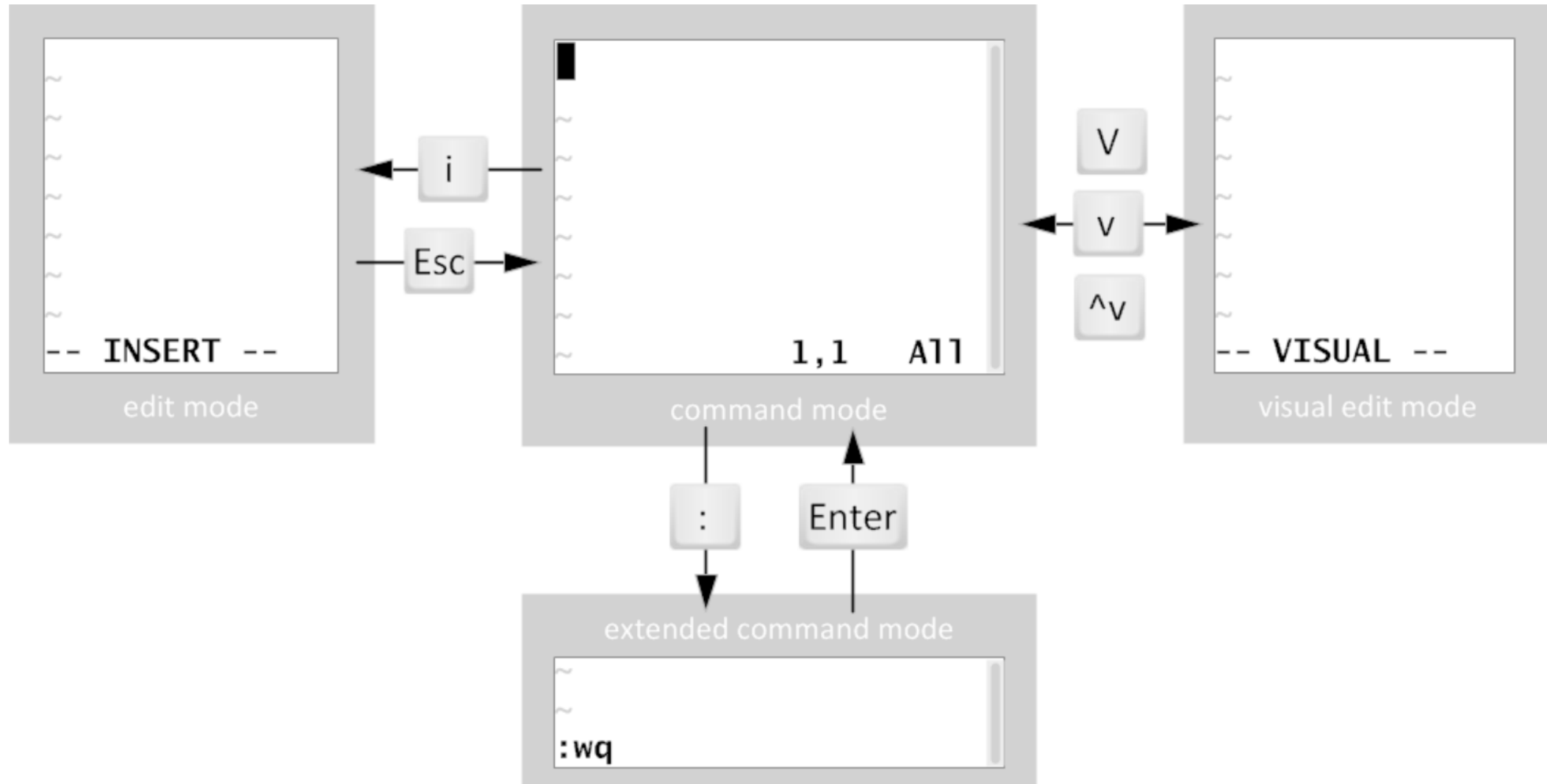
Настоящий курс не ставит своей задачей углубленное изучение Vim и знакомит слушателей с его базовыми возможностями

Режимы работы Vim

- Командный режим (по умолчанию при запуске Vim)
 - Перемещение по тексту
 - Копировать/вырезать/вставить
- Режим редактирования
 - Ввод текста с клавиатуры
 - Insert – переключение режимов вставки/замены текста
- Визуальный режим
 - Выделение групп символов и операции над ними
 - v – посимвольное выделение
 - Shift+v – строочное выделение
 - Ctrl+v – выделение блоков
- Расширенный командный режим
 - Запись в файл (сохранение изменений)
 - Выход из Vim

Режимы работы Vim

Переключение между режимами



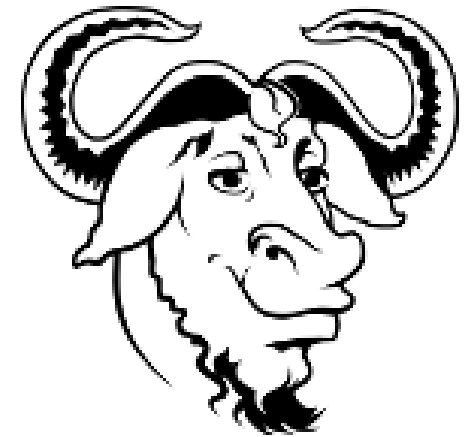
Базовые приемы работы в Vim

- 🔄 Откройте файл, запустив Vim
- 🔄 Просмотрите текст, найдите место, которое нужно изменить
- 🔄 Перейдите в режим редактирования (`i` или `Insert`)
 - Добавьте или измените текст
- 🔄 Закончив редактирование вернитесь в командный режим (`Esc`)
- 🔄 При необходимости используйте отмену последних изменений (`u`)
- 🔄 Для перемещения текста
 - Поместите курсор в начало текста и перейдите в визуальный режим (`v`)
 - Выделите текст
 - Используйте копирование (`y`) и вставку (`p`) – `yank/put`
- 🔄 Для завершения редактирования файла
 - Перейдите в расширенный командный режим (`:`)
 - Сохраните изменения (`w`)
 - Выйдите из Vim (`q`)
 - Выход с сохранением (`wq`), выход без сохранения (`q!`)

- Перенаправление вывода в файл или программу
- Редактирование текстовых файлов в командной строке
- **Поиск и фильтрация текста: `grep`**
- Регулярные выражения
- Поточковые редакторы текста: `sed` и `awk`
- Изменение настроек среды командной строки

Утилита grep

- Создана в 1973 г., входит в большинство дистрибутивов Linux
- Находит во вводе строки, удовлетворяющие заданному шаблону (регулярному выражению), и направляет их в стандартный вывод
 - search Globally for lines matching the Regular Expression, and Print them (**GREP**)
- Получает текст из стандартного ввода или файла
 - Если часть строки совпадает с заданным шаблоном, вся строка направляется в вывод
 - Символ новой строки используется как разделитель строк
- Возможен поиск по нескольким файлам
- Вывод может дополняться, например, номерами строк или именами файлов, где найдены совпадения
- Основные применения:
 - Поиск информации в текстовых файлах
 - Фильтрация вывода других команд (конвейеры)



Запуск grep

● Стандартное использование grep:

grep [параметры...] [шаблоны] [файлы...]

- Вывод направляется в стандартный канал вывода и может быть перенаправлен
- Если не указано имя входного файла, данные для обработки берутся из стандартного канала ввода

```
$ grep 'hello' input.txt > output.txt  
$ grep 'hello' < input.txt > output.txt  
$ cat input.txt | grep 'hello' > output.txt
```

Примеры команд эквивалентны (находит в файле `input.txt` строки, содержащие слово `'hello'`, и записывает их в файл `output.txt`)

● Шаблон – регулярное выражение

- Берется из текста команды или файла
 - Если нет параметра **-e(-f)**, первый аргумент команды воспринимается как шаблон
- Может быть задано несколько шаблонов (в тексте команды и/или файлах)
 - Выводятся строки, удовлетворяющие хотя бы одному шаблону

Запуск grep

Управление шаблонами для сравнения

- 🔗 Задать один или несколько шаблонов в тексте команды *
- e pattern (--regex=patterns)
- 🔗 Задать один или несколько шаблонов в файле *
- f file (-- file = file)
- 🔗 Сравнение без учёта регистра
- i (-y, --ignore-case)
- 🔗 Инверсия шаблона (выводятся строки без совпадений)
- v (--invert-match)

```
$ grep 'hello' input.txt  
$ grep -e 'hello' -e 'world' input.txt  
$ grep -e 'hello' -f patterns.txt input.txt
```

* - Несколько шаблонов для сравнения можно задать через несколько параметров
-e и/или -f (Альтернатива: разделить шаблоны в тексте команды и файлах символом новой строки)

Запуск grep

Управление выводом

- Вывести количество строк, удовлетворяющих шаблону *
-c (--count)
- Включить цветовую подсветку вывода команды
--color (--colour)
 - По умолчанию красным выделяются совпадающий с шаблоном фрагмент строки
- Вывести имена файлов с совпадением / без совпадения *
-l / -L (--files-with-matches / --files-without-matches)
- Ограничить вывод заданным числом строк
-m [число] (--max-count=[число])

```
$ echo 'Hello, World!' | grep -i --color 'hello'  
'Hello, World!'
```

* - Строки, удовлетворяющие шаблону, не выводятся

Запуск grep

Управление выводом контекста

- Вывести [число] строк, следующих за строкой с совпадением
-A [число] (--after-context=[число])
- Вывести [число] строк, предшествующих строке с совпадением
-B [число] (--before-context=[число])
- Вывести [число] строк, идущих до и после строки с совпадением
-C [число] (-[число], --context=[число])
- Выводить номера строк (нумерация с 1)
-n (--line-number)

<u>input.txt</u> Line1 Line2 Line3 Line4 Line5 Line6 Line7

<pre>\$ grep -in -e 'line2' -e 'line6' -C 1 input.txt 1-Line1 2:Line2 3-Line3 -- 5-Line5 6:Line6 7-Line7</pre>
--

- Перенаправление вывода в файл или программу
- Редактирование текстовых файлов в командной строке
- Поиск и фильтрация текста: `grep`
- **Регулярные выражения**
- Поточковые редакторы текста: `sed` и `awk`
- Изменение настроек среды командной строки

Регулярные выражения (Regular expressions)

- Регулярные выражения – формальный язык, для обработки текста, включая поиск фрагментов текста и осуществления манипуляций над ними, основанный на использовании метасимволов
 - Метасимволы позволяют описывать шаблоны, которым будут соответствовать строки с заданными свойствами, например, по набору и количеству символов, их последовательности и т.п.
- Типичной сценарий использования регулярных выражений – поиск строк, соответствующих шаблону, с последующим их выводом или заменой на заданный текст
- Большинство программ GNU для работы с текстом поддерживают регулярные выражения и могут получать их в качестве аргументов при запуске
 - Синтаксис Perl Compatible RE поддерживается при наличии в системе библиотеки PCRE

Базовый синтаксис регулярных выражений

Элемент	Описание	Пример	Совпадения
СИМВОЛ	Представляет указанный символ	a	'a'
.	Один любой символ	a.c	'abc', 'a1c', 'aZc'
*	Ноль или несколько предшествующих элементов	a*	'', 'a', 'aa', 'aaa'
^	Начало строки	^abc	'abcx', но не 'xabc'
\$	Конец строки	abc\$	'xabc', но не 'abcx'
+	Один или несколько предшествующих элементов	a+	'a', 'aaa', но не ''
?	Ноль или один предшествующий элемент	a?	'', 'a', но не 'aa'
[list]	Символ из списка	a[bc]d	'abd', 'acd'
[^list]	Символ не из списка	a[^bc]d	'axd', но не 'abd'
[x-y]	Символ из диапазона от x до y*	a[b-d]e	'ade', 'ace'
{i}	i повторений предшествующего элемента	a{2}	'aa', но не 'aaa'
{i,j}	От i до j повторений предшествующего элемента*. * включая границы	a{1,3}	'aa', 'aaa'
{i,}	i и более повторений предшествующего элемента	a{3,}	'aaaa', но не 'aa'
{,j}	j и менее повторений предшествующего элемента	a{,3}	'aa', но не 'aaaa'

Примеры использования регулярных выражений

Примеры использования:

- Поиск слова "hello" в строке:

```
$ grep "hello" file.txt
```

- Поиск слова, начинающегося с буквы "h":

```
$ grep "^h" file.txt
```

- Поиск слова, заканчивающегося на букву "o":

```
$ grep "o$" file.txt
```

- Поиск слова, содержащего букву "e" и заканчивающегося на букву "o":

```
$ grep "e.*o$" file.txt
```

- Замена слова "hello" на "hi":

```
$ grep 's/hello/hi/g' file.txt
```

- Поиск всех чисел в строке:

```
$ grep "[0-9]+" file.txt
```

- Поиск всех слов, начинающихся на букву "a" или "b":

```
$ grep "^(a|b)" file.txt
```

- Поиск всех слов, содержащих букву "e", заканчивающихся на букву "o", но не начинающихся на букву "h":

```
$ grep "^(?!h).e.o$" file.txt
```

Базовый синтаксис регулярных выражений

Элемент	Описание	Пример	Совпадения
(RegEx)	Вложенное выражение	(abc)+	'abcabc'
RegEx1 RegEx2	Текст должен удовлетворять или первому или второму выражению	(abc) (xyz)	'abc', 'xyz'
RegEx1RegEx2	Текст должен удовлетворять конкатенации двух выражений	(abc)(xyz)	'abcxyz'
\число	Ссылка на вложенное выражение по его номеру по порядку	(abc)(xyz)\1	'abcxyzabc'
\n	Символ новой строки	a\nb	'a b'
\символ	Экранированный символ, воспринимается как обычный символ, а не метасимвол	a\\$	'a\$'
\s	Пробел (пробел, табуляция и т.п.)	a\s c	'a c', но не 'abc'
\S	Не-пробел (любой символ кроме пробела)	a\S c	'abc', но не 'a c'

Примеры использования регулярных выражений

🔍 Найти все строки, содержащие подстроку 'Hello, World!'

- В точности

```
$ grep 'Hello, World!' input.txt
```

- С заглавными или строчными буквами

```
$ grep '[Hh]ello, [Ww]orld!' input.txt
```

- С произвольным количеством пробелов

```
$ grep 'Hello, \+World!' input.txt
```

- Запятая необязательна

```
$ grep 'Hello,\? World!' input.txt
```

- В конце от одного до трех восклицательных знаков

```
$ grep 'Hello, World!\{1,3\}' input.txt
```

- Повторяющуюся дважды через любой пробел

```
$ grep '\(Hello, World!\)\s\1' input.txt
```

- Строго в начале строки

```
$ grep '^Hello, World!' input.txt
```

- Перенаправление вывода в файл или программу
- Редактирование текстовых файлов в командной строке
- Поиск и фильтрация текста: `grep`
- Регулярные выражения
- **Потоковые редакторы текста: `sed` и `awk`**
- Изменение настроек среды командной строки

Потоковые редакторы текста

- Применяют различные predetermined текстовые преобразования к последовательному потоку текстовых данных
- Обычно считывают данные из файла построчно и применяют преобразования к каждой строке, направляя результат в стандартный вывод (может быть перенаправлен в файл)
- Для описания преобразований используются команды специального языка, который специфичен для каждого редактора, и регулярные выражения
- В отличие от обычных редакторов (например, Vim) используют подход «инвертированный» по отношению к тексту
 - Обычные редакторы загружают весь текст документа, а затем применяют к нему команды последовательно или работают в интерактивном режиме
 - Потоковые редакторы вначале загружают в себя набор команд, а затем применяют его к каждой строке текста
 - В каждый момент времени в памяти находится только одна строка, что позволяет эффективно работать с очень большими текстовыми файлами

Потоковый редактор sed

- Stream Editor создан в 1974 г.
- Входит в большинство дистрибутивов Linux
- Использует собственный набор команд, основанный на ed, и регулярные выражения
 - Является Тьюринг-полным языком
- Удобен для автоматизации внесения изменений в текстовые файлы, например, файлы конфигурации или сценарии
- В настоящем курсе рассматривается GNU sed, имеющий новые возможности по сравнению со стандартной версией (POSIX)
 - Замещающая правка позволяет вносить изменения в файлы-источники
 - Расширен синтаксис регулярных выражений и др.

Настоящий курс не ставит своей задачей углубленное изучение sed и знакомит слушателей с его базовыми возможностями

Запуск sed (1/3)

● Стандартное использование sed:

`sed SCRIPT INPUTFILE...`

- Вывод направляется в стандартный канал вывода и может быть перенаправлен
- Если не указано имя входного файл или оно задано как '-', данные для обработки берутся из стандартного канала ввода

```
$ sed 's/hello/world/' input.txt > output.txt  
$ sed 's/hello/world/' < input.txt > output.txt  
$ cat input.txt | sed 's/hello/world/' - > output.txt
```

Примеры команд эквивалентны (заменяет все вхождения слова 'hello' на 'world' в тексте взятом из input.txt и записывает результат в output.txt)

● Использование sed с замещающей правкой (параметр -i)

- Входной файл будет перезаписан выводом sed

```
$ sed -i 's/hello/world/' file.txt
```

Запуск sed (2/3)

- По умолчанию sed записывает в вывод весь полученный ввод (кроме удаленных/измененных данных)

- Параметр -n блокирует вывод
- Команда p выводит указанные строки

```
$ sed -n '45p' file.txt
```

Выводит только строку номер 45 из исходного файла

- Если указать несколько входных файлов, sed воспринимает их как один непрерывный поток текста

- Параметр -s указывает, что файлы должны обрабатываться по отдельности

```
$ sed -n '1p ; $p' one.txt two.txt three.txt  
$ sed -n -s '1p ; $p' one.txt two.txt three.txt
```

Первая команда выводит первую строку первого файла (one.txt) и последнюю строку последнего файла (three.txt), вторая – первую и последнюю строку каждого файла

Запуск sed (3/3)

- Без параметров `-e` или `-f` первый аргумент воспринимается `sed` как сценарий, а последующие аргументы – как входные файлы
 - При наличии параметра `-e` или `-f` все аргументы воспринимаются как имена входных файлов
- Параметры `-e` или `-f` служат для задания сценария
 - `-e (--expression)` – позволяет задать сценарий непосредственно в команде
 - `-f (--file)` – задает имя файла, откуда нужно прочитать сценарий
 - Могут использоваться одновременно и многократно
 - Применяемые сценарий будет результатом конкатенации заданных сценариев

```
$ sed 's/hello/world/' input.txt > output.txt

$ sed -e 's/hello/world/' input.txt > output.txt
$ sed --expression='s/hello/world/' input.txt > output.txt

$ echo 's/hello/world/' > myscript.sed
$ sed -f myscript.sed input.txt > output.txt
$ sed --file=myscript.sed input.txt > output.txt
```

Сценарии sed

Общая структура

- Сценарий (программа) sed состоит из одной или нескольких команд заданных при запуске sed
 - Далее везде подразумевается результирующий сценарий, полученный сложением всех сценариев, заданных при запуске
- Синтаксис команды:
[адрес]X[параметры], где X – однобуквенная команда sed
 - Если [адрес] задан, команда применяется только к соответствующим строкам
 - [параметры] – специфичны для команд
- Адрес может быть задан как
 - Номер строки
 - Диапазон номеров строк
 - Регулярное выражение
 - Формальный язык, для описания текстовых шаблонов, основанный на использовании символов и метасимволов

Сценарии sed

Адрес

Адрес	Описание	Пример	
число	Номер строки (нумерация с 1)	5	Пятая строка
\$	Последняя строка		
старт~шаг	Все строки, начиная со старт+шаг, с заданным шагом	0~2	Все четные строки
/regex/	Все строки, содержащие подстроку, соответствующую регулярному выражению	/hello/	Все строки со словом 'hello'
старт, стоп	Все строки с номерами от старт до стоп включительно	2,5	Строки со второй по пятую
адрес !	Инвертирует адрес	5!	Все строки кроме пятой

```
$ seq 10 | sed -n '0~4p'
4
8
```

```
$ sed -n '/bash$/p' /etc/passwd
```

Сценарии sed

Основные команды

Команда	Описание
a	Добавить текст после строки
c	Заместить строку текстом
d	Удалить строку
i	Вставить текст перед строкой
p	Напечатать строку (используется при отключенном выводе, параметр -n)
s	Заменить часть строки текстом (см. следующий слайд)

```
$ seq 3 | sed '2a hello'
1
2
hello
3
```

```
$ seq 10 | sed '2,9c hello'
1
hello
10
```


Сценарии sed

Команда s (substitute)

Наиболее часто используемая команда sed

- Заменяет текст, удовлетворяющий шаблону, на другой текст

Синтаксис команды S:

`s/regex/replacement/flags`

- `regex` – регулярное выражение, задающее шаблон
- `replacement` – текст, на который осуществляется замена
- `flags` – флаги, влияющие на процесс замены
- Символ `'/'`, разделяющий части команды, может быть заменён на любой другой одиночный символ
 - Символ-разделитель надо обязательно экранировать, если он встречается внутри `regex` или `replacement` частей команды
- Для экранирования служебных символов используется символ `'\'`

```
$ sed -i 's/http:\\\\//ftp:\\\\//g' file.txt
$ sed -i 's|http:\\\\|ftp:\\\\|g' file.txt
```

Потоковый редактор awk

● AWK (аббревиатура от первых букв фамилий разработчиков)

- Сценарный язык, управляемый данными (data driven) для поиска в тексте фрагментов, соответствующих заданным шаблонам, и совершения над ними различных действий
- Одноименная программа, входящая в большинство дистрибутивов Linux, для выполнения сценариев, написанных на awk

● Сценарии awk состоят из наборов *правил*

- Синтаксически правило – это *шаблон*и следующее за ним *действие*
- Правила обычно пишутся с новой строки
- Действия заключаются в фигурные скобки, чтобы отделить их от шаблонов

```
pattern { action }  
pattern { action }
```



Настоящий курс не ставит своей задачей углубленное изучение awk и знакомит слушателей с его базовыми возможностями

Запуск awk

● Стандартное использование awk:

awk SCRIPT INPUTFILE...

awk -f SCRIPT-FILE INPUTFILE...

- Вывод направляется в стандартный канал вывода и может быть перенаправлен
- Если не указано имя входного файла, данные для обработки берутся из стандартного канала ввода пока не будет получен символ "конец файла"

```
$ awk '{ print }' input.txt > output.txt
```

```
$ awk 'BEGIN { print "Don\'t Panic!" }'  
-| Don't Panic!
```

```
$ awk '{ print }'  
Some text  
-| Some text  
more text  
-| more text  
Ctrl-d
```

Как awk работает с входными данными

- Весь входной поток данных разбивается на **записи**
 - Запись – фрагмент текста, ограниченный символами «конец строки»
 - Разделитель записей может быть переопределен (переменная RS)
- Каждая запись разбивается на **поля**
 - Поле – фрагмент текста, ограниченный пробелами
 - Пробелом считается любое количество символов «пробел», «табуляция», «конец строки» и т.п.
 - Разделитель полей может быть переопределен (параметр -F или переменная FS)
 - Возможно разбиение на поля заданной длины
- Если запись удовлетворяет шаблону в блоке действие к её полям можно обратиться используя конструкцию \$N (где N – номер поля, нумерация с 1)
 - \$1 – первое поле
 - \$NF – последнее поле
 - \$0 – вся запись целиком

Шаблоны awk

● Шаблоны управляют выполнением правил

- Если в записи найдено совпадение с шаблоном выполняется действие

● Типы шаблонов

- Регулярное выражение (/regexp/)
 - Действие выполняется, если в записи найдено совпадение с регулярным выражением
- Выражение
 - Действие выполняется, если значение выражение отлично от 0 (для численных выражений) или не пусто (для строк)
- Диапазон (шаблон1, шаблон2)
 - Действие выполняется для записей начиная с совпадения первого шаблона до совпадения второго шаблона
- BEGIN, END, BEGINFILE, ENDFILE
 - Выполняют действие в начале обработки входных данных или после ее завершения (*FILE – используются при пофайловой обработке)
- Пустой шаблон (шаблон не задан)
 - Действие выполняется для всех записей

- Действия определяют, что делать при совпадении шаблона
 - Если в правиле нет действия, применяется действие по умолчанию и запись направляется в стандартный канал вывода
- Действия описываются следующими конструкциями языка `awk`
 - Выражение
 - Вычисление значений математических и строковых выражений с присвоением результата переменной, вызов функций
 - Управляющие операторы
 - Условные переходы (`if-else`, `switch`), циклы (`for`, `while`, `do-while`), прерывания (`break`, `continue`, `exit`)
 - Блоки
 - Блоки обозначаются фигурными скобками `{...}`
 - Операторы ввода
 - Явно получают данные из канала ввода (`getline`, `next`, `nextfile`)
 - Операторы вывода
 - Пишут данные в канал вывода (`print`, `printf`)

Использование awk (1/2)

- Агрегирование данных, полученных от других программ или из файлов, построение отчетов:

- Вывести строки длиннее 80 символов

```
$ awk 'length($0) > 80' input.txt
```

- Вывести количество символов в самой длинной строке

```
$ awk '{ if (length($0) > max) max = length($0) }  
END { print max }' data
```

- Посчитать суммарный размер файлов в директории в килобайтах

```
$ ls -l files | awk '{ x += $5 }  
END { print "total K-bytes:", x / 1024 }'
```

- Вывести отсортированный список имен пользователей

```
$ awk -F: '{ print $1 }' /etc/passwd | sort
```

- Вывести суммарный размер файлов, измененных в ноябре

```
$ -ls | awk '$6 == "Nov" { sum += $5 }  
END { print sum }'
```

Использование awk (2/2)

● Редактирование данных на уровне отдельных полей

- awk никогда не изменяет входные файлы, изменённые данные могут быть записаны в новый файл

- Уменьшить значение второго поля во всех строках на 10

```
$ awk '{ $2 = $2 - 10; print $0 }' input.txt > output.txt
```

- Добавить в конец каждой строки новое поле, содержащее сумму предшествующих полей

```
$ echo 1 2 3 4 | awk '{ $5 = $1+$2+$3+$4; print $0 }'  
1 2 3 4 10
```

- Поменять разделитель полей

```
$ echo a b c d | awk '{ OFS = ":"; $2 = $2; print $0 }'  
a:b:c:d
```

- OFS – переменная, определяющая разделитель полей в выводе
- Если запись не изменялась, то она не будет перестроена с учетом значения OFS, требуется любое изменение, например присвоение поля самому себе

- 🔄 Перенаправление вывода в файл или программу
- 🔄 Редактирование текстовых файлов в командной строке
- 🔄 Поиск и фильтрация текста: `grep`
- 🔄 Регулярные выражения
- 🔄 Поточковые редакторы текста: `sed` и `awk`
- 🔄 **Изменение настроек среды командной строки**

Переменные в bash

- Переменная командной строки служит для хранения значения, на которое можно ссылаться в командах по имени переменной
 - Имя переменной может включать строчные и заглавные латинские буквы, цифры и символ подчеркивания (`_`)
 - Значение переменной – строка
- Переменная объявляется в момент присвоения ей значения
 - Переменная доступна в сессии, где была объявлена. В другой сессии переменная может иметь другое значение или вообще не быть объявлена

```
$ COUNT=40  
$ first_name=John  
$ file1=/tmp/abc  
$ _ID=MOF01
```

- Переменная командной строки может быть экспортирована как переменная окружения
 - Программы имеют автоматический доступ к переменным окружения, объявленным в сессии, откуда программа была запущена

Доступ к значениям переменных

- Значение переменной можно получить в результате ее раскрытия

- Символ `$` добавляется в начало имени переменной

```
$ COUNT=40
$ echo COUNT
COUNT
$ echo $COUNT
40
```

- Переменные помогают сохранять значения полученные от программ и многократно использовать их

```
$ FILE_NAME=$(ls -la | awk '/bash/ {print $9; exit }')
$ echo $FILE_NAME
.bash_history
$ cat $FILE_NAME
...
```

- Полный список переменных (и функций), объявленных в сессии можно получить командой **set**

Настройка bash при помощи переменных

- Специальные переменные командной строки влияют на ее работу
 - Получают значения из настроек в момент запуска командной строки
 - Могут быть изменены в рамках сессии

HISTFILE – определяет путь для сохранения истории командной строки (доступна по команде **history**)

- По умолчанию: ~/.bash_history

HISTFILESIZE – определяет количество последних команд в истории

PS1 – определяет вид приглашения командной строки

```
user@host:~$ PS1="myprompt\$ "  
myprompt$ PS1="[ \u@\h \W]\$ "  
[user@host ~]$
```

- Список переменных, используемых bash, доступен в **man bash**

Настройка программ при помощи переменных окружения

- Командная строка предоставляет окружение программам, которые из неё запущены. Окружение содержит данные о:
 - Текущей рабочей директории
 - Параметрах командной строки, с которыми запущена программа
 - Значениях переменных окружения
- Переменная командной строки может быть экспортирована как переменная окружения командой **export**

```
$ EDITOR=vim  
$ export EDITOR
```

```
$ export EDITOR=vim
```

- Переменные командной строки недоступны программам
- Полный список переменных окружения, объявленных в сессии можно получить командой **env**

Стандартные переменные окружения (примеры)

HOME – содержит путь к домашней директории пользователя, от имени которого запущена сессия

- Используется программами как путь для файлов по умолчанию
- Параметрах командной строки, с которыми запущена программа
- Значениях переменных окружения

LANG – определяет используемую локаль

```
$ date
Пт апр  7 17:19:08 MSK 2023
$ export LANG=en_EN.UTF-8
$ date
Fri Apr  7 17:20:41 MSK 2023
```

EDITOR – определяет редактор по умолчанию

```
$ export EDITOR=nano
```

Переменная окружения PATH

● Содержит список директорий, записанный через двоеточие (:)

- При запуске команды, интерпретатор командной строки ищет соответствующий ей исполняемый файл в директориях из этого списка, перебирая их по порядку, и запускает первый найденный файл

```
$ echo $PATH  
/usr/local/bin:/usr/bin:/bin:/usr/local/games:/usr/games
```

- Например, при запуске команды **ls** соответствующий файл будет найден в **/bin**

● Чтобы указать новые директории для поиска исполняемых файлов, пути к ним можно добавить в конец списка

```
$ export PATH=${PATH}:/mydir  
$ echo $PATH  
/usr/local/bin:/usr/bin:/bin:/usr/local/games:/usr/games:/mydir
```

- Фигурные скобки отделяют имя переменной от остального текста

Автоматическое назначение значений переменных

- При запуске командной строки выполняется несколько сценариев, устанавливающих значения переменных в текущей сессии
 - За настройки для интерактивных сессий пользователя отвечает сценарий `~/.bashrc`, в которой могут быть добавлены объявления переменных
 - Пример содержимого файла:

```
# .bashrc

# Source global definitions
if [ -f /etc/bashrc ]; then
. /etc/bashrc
fi

# User specific environment
PATH="$HOME/.local/bin:$HOME/bin:$PATH"
export PATH

# User specific aliases and functions
export EDITOR=nano
```


Удаление и отмена экспорта переменных

- Команда **unset** удаляет ранее объявленную переменную
 - При обращении к удаленной переменной мы получим пустое значение (аналогично обращению к необъявленной переменной)
- Команда **export -n** отменяет экспорт переменной, не удаляя ее

```
$ MY_PATH = /home/user/mydir  
$ export MY_PATH
```

```
$ export -n MY_PATH  
$ echo $MY_PATH  
/home/user/mydir
```

```
$ unset MY_PATH  
$ echo $MY_PATH  
  
$
```

В данной лекции были рассмотрены следующие темы:

- 🔄 Перенаправление вывода в файл или программу
- 🔄 Редактирование текстовых файлов в командной строке
- 🔄 Поиск и фильтрация текста: `grep`
- 🔄 Регулярные выражения
- 🔄 Поточковые редакторы текста `sed` и `awk`
- 🔄 Изменение настроек среды командной строки